

Emotion Recognition System

Project ID:22MPG111

Final Review

Group Members

RA1811003030377 Arpit Khare

RA1811003030382 Archismaan Banerjee

RA1811003030401 Rajashvi Srivastava

RA1811003030405 Aditi Anand

Supervised By:

Mr. Amit Sharma

Assistant Prof.

Department of Computer Science & Engineering
Faculty of Engineering & Technology
SRM Institute of Science & Technology



Table of Contents I

- 1 Abstract
- 2 Objectives
- 3 Literature Survey
- 4 Overall Design for Proposed System
- 5 Techniques to be used
- 6 Algorithms
- 7 Database/ Dataset Specification
- 8 Methodology
- 9 Experimental Results
- 10 Performance Evaluation
- 11 Comparison with existing system
- 12 References

Abstract

In today's world, the Human face uses expressions to convey a lot of information visually, hence Emotion Recognition System plays a crucial role in the area of human-machine interaction. Our facial emotions are expressed through the activation of specific sets of facial muscles. These sometimes subtle, yet complex, signals in an expression that often contains an abundant amount of information about our state of mind. Through emotion recognition, we can measure the effects that content and services have on the users through an easy and low-cost procedure. We designed a deep learning neural network that gives machines the ability to make inferences about our emotional states. The main objective of our project is that we applied various deep learning methods like convolutional neural networks to identify general human emotions.

Objectives

Feeling acknowledgment assumes an essential part in the period of Artificial insight and the Internet of things. It offers a huge extension to human PC collaboration, mechanical technology, medical care, bio metric security, and social demonstration. Feeling acknowledgment frameworks perceive feelings from facial articulations, text information, body developments, voice, cerebrum, or heart signals. Alongside fundamental feelings, disposition, command over feelings, and force of initiation of feeling can likewise be inspected for examining opinions. This idea distinguishes different directed and unaided AI strategies for including extraction and feeling characterization. A similar investigation has likewise been made of different machine learning calculations utilized in referred to papers. It tells the extension and uses of programmed feeling acknowledgment frameworks in different fields. This idea additionally talks about different boundaries to expand the precision, security, and productivity of the framework.

The significant targets and thoughts include:

1. Locating faces in the scene (e.g., in an image; this step is also referred to as face detection),
2. Extracting facial features from the detected face region (e.g., detecting the shape of facial components or describing the texture of the skin in a facial area; this step is referred to as facial feature extraction),
3. Analyzing the motion of facial features and/or the changes in the appearance of facial features and classifying this information into some facial-expression-interpretative categories such as facial muscle activation's like smile or frown, emotion (affect)categories like happiness or anger, attitude categories like (dis)liking or ambivalence, etc. [7]

Literature Review

:

Author Name	Title	Source	Findings
Byoung Chul Ko	A Brief Review of Facial Emotion Recognition Based on Visual Information	Department of Computer Engineering, Keimyung University, Daegu 42601, Korea; niceko@kmu.ac.kr;	The classification algorithms used in conventional FER include SVM, Adaboost, and random forest ;by contrast, deep-learning-based FER approaches highly reduce the dependence on face-physics-based models and other pre-processing techniques by enabling "end-to-end" learning in the pipeline directly from the input images.

Literature Review II

Ronak Kosti, Jose M. Alvarez, Adria Recasens, Agata Lapedriza	Emotion Recognition in Context	Universitat Oberta de Catalunya Data61 / CSIRO Massachusetts Institute of Technology	They proposed a CNN model for the task of emotion estimation in context. The model is based on state-of-the- art techniques for visual recognition and provides a benchmark on the proposed problem of estimating emotional states in context.
Shashidhar G. Koolagudi K. Sreenivasa Rao	Emotion recognition from speech: a review	© Springer Science+ Business Media, LLC 2011	researchers use non- linear classifiers always at the cost of complexity and computational time. However systematic approach based on nature of speech features is required while choosing the pattern classifiers for emotion recognition.

Literature Review III

Bjorn Schuller, Gerhard Rigoll, and Manfred Lang	HIDDEN MARKOV MODEL-BASED SPEECH EMOTION RECOGNITION	Institute for Human- Computer Communication Technische Universitat Miinchen	The two introduced methods proved both capable of a rather reasonable model for the automatic recognition of human emotions in speech.
Alvin I. Goldmana, Chandra Sekhar Sripada	Simulationist models of face-based	Center for Cognitive Science, Rutgers University, P.O. Box 1179, Piscataway, NJ 08855-1179, USA	Four alternative models have been explored: a generate-and-test model, a reverse simulation model, a variant of the reverse simulation model that employs an "as if" loop, and an unmediated resonance model.

Literature Review IV

Moataz El Ayadi, Mohamed S. Kamel, Fakhri Karray b	Survey on speech emotion recognition: Features, classification schemes,	Engineering Mathematics and Physics, Cairo University, Giza 12613, Egypt	The performance of current stress detectors still needs significant improvement. The average classification accuracy of speaker-independent speech emotion recognition systems is less than 80% in most of the proposed techniques.
Suraj Tripathi1, Abhay Kumar1*, Abhiram Ramesh1*, Chirag Singh1*, Promod	Deep Learning based Emotion Recognition System Using	Samsung R&D Institute India – Bangalore	Speech features based 2D CNN model provides better ac-curacy relative to state-of-the-art results, which further improves when combined with text.
Emily Mower, Student Member, IEEE, Maja J Mataric', Senior Member, IEEE, and Shrikanth Narayanan, Fellow, IEEE	A Framework for Automatic Human Emotion	IEEE TRANSACTIONS ON AUDIO, SPEECH, AND LANGUAGE PROCESSING, VOL. 19, NO. 5, JULY 2011	The utility of emotion- based components for representation rather than data-driven clusters as the relevant components in the profile construction. These analyses will provide further evidence regarding the efficacy of profile-based representations of emotion.

Literature Review V

Ryoko TOKUHISA, Kentaro INUI, Yuji MATSUMOTO	Emotion Classification Using Massive Examples Extracted from the Web	Proceedings of the 22nd International Conference on Computational Linguistics (Coling 2008)	In this paper, we have addressed the issue of emotion classification assuming its potential applications to be human-computer dialog system including active-listening dialog
Roddy Cowie, Ellen Douglas-Cowie, Nicolas Tsapatsoulis, George Votsis, Stefanos Kollias, Winfried Fellenz, and John G Taylor.	Emotion Recognition in Human-Computer Interaction	IEEE SIGNAL PROCESSING MAGAZINE JANUARY 2001	In the context of automatic emotion recognition, understanding the nature of emotion is not an end in itself. It matters mainly because ideas about the nature of emotion shape the way emotional states are described. They imply that certain features and relationships are relevant to describing an emotional state, distinguishing it from others, and determining whether it qualifies as an emotional state at all

Overall Design for Proposed System

- SOFTWARE REQUIREMENT :

The project is developed using python, we have used Python 3.6.5

Hardware Interfaces:

1. Processor : Intel CORE i5 5th gen processor with minimum 2.9 GHz speed.
2. RAM : Minimum 4 GB.
3. Hard Disk : Minimum 250 GB

Software Interfaces:

1. Microsoft Word 2003
2. Database Storage : Microsoft Excel
3. Operating System : Windows10

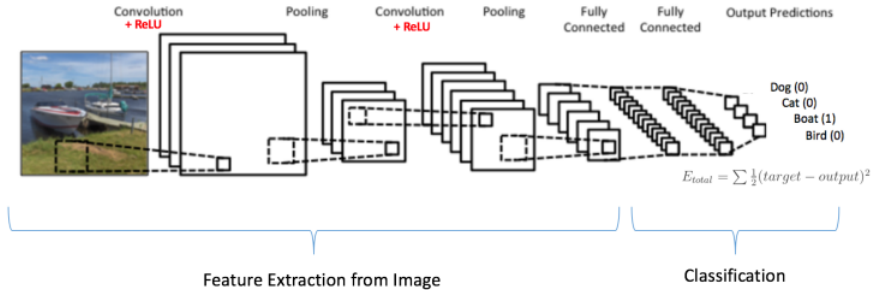
● PLANNING :

The steps we followed when developing this project are:

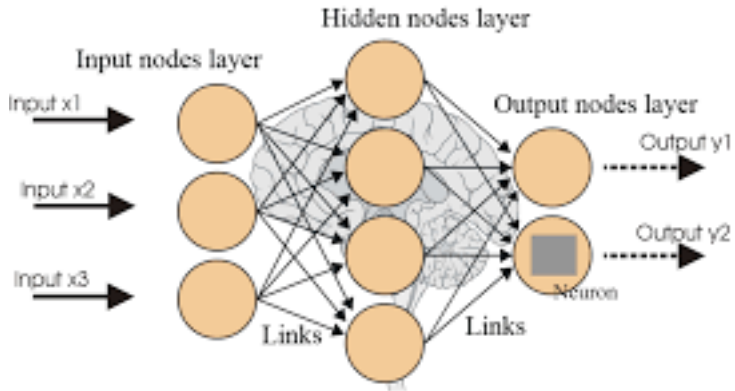
1. Problem statement analysis.
2. Collection of set requirements
3. Project feasibility analysis.[5]
4. Development of a standard structure.
5. Journals about past activities related to this field.
6. Choosing a method to improve the algorithm.
7. Various good and bad analyzes.
8. Initiate project development
9. Installation of software and PIP packages.
10. Improving algorithm.
11. Manual algorithm analysis.
12. Coding according to the advanced algorithm in PYTHON.

We built this project based on a iterative waterfall model:

The Division of layers throughout



Artificial Neural Network



- **UML** Business analysts, software architects, and developers use UML to define, specify, create, and record current or new business processes, as well as the structure and behaviour of software system artefacts. UML may be used in a wide range of applications (e.g., banking, finance, internet, aerospace, healthcare, etc.) It's compatible with all major object and component programme development methodologies, as well as a variety of implementation platforms. When it comes to modelling them with Machine Learning, the technique you choose has a big impact. When a category model of human emotion is selected, a classifier is almost certainly built, and texts and images are labelled with human emotions such as "happy," "sad," and so on. When using a dimensional model, however, outputs must be on a sliding scale.

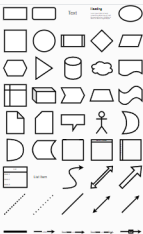


Untitled Diagram.drawio.png

File Edit View Arrange Extras Help

Unsaved changes. Click here to save.

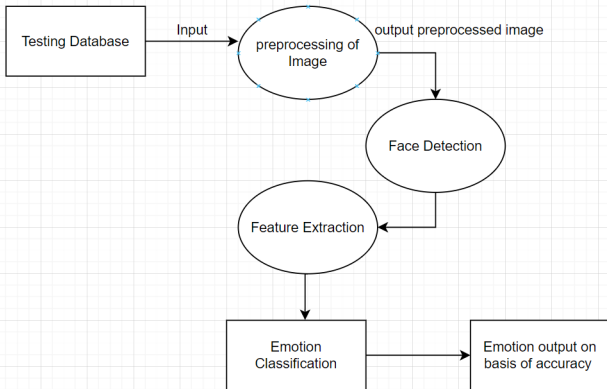
145%



Misc



+ More Shapes...

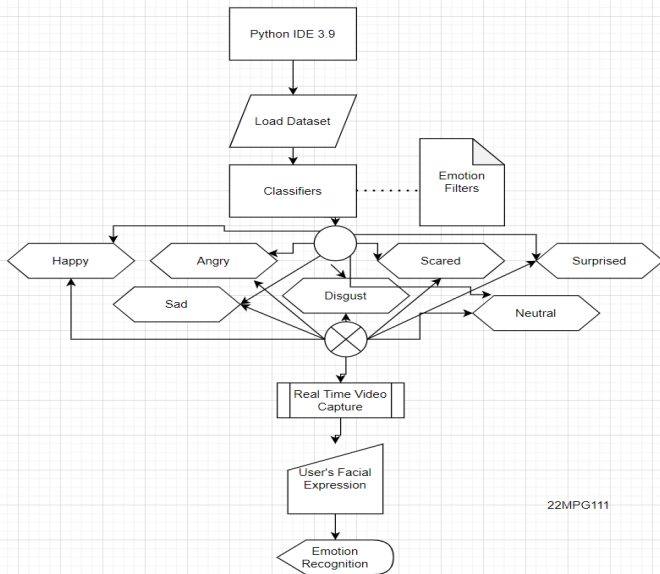


Page-1



INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s of UGC Act, 1956

Emotion Classification



22MPG111

Techniques to be used

- Face Registration
- Haar Features
- Integral Images
- Adaboost
- Cascading
- Haar Cascade Classifier in OpenCv
- Artificial Neural Networks
- Deep Convolutional Neural Networks
- Convolutional Layers
- Pooling Layers
- Fully Connected Layers
- Training

Algorithms

Step 1: Collection of image data set. (This time we use the FER2013 website 35887 48x48-pixel gray face images labeled with 7 categories of emotions: anger, disgust, fear, joy, sadness, surprise, and neutrality.

Step 2 :Pre-image processing.[1]

Step 3 :Face detection in each photo.

Step 4 :The cut face is converted to gray images.

Step 5 : The pipe ensures that the whole image can be inserted into the input layer as a program (1, 48, 48) numpy.

Step 6 :Similar numpy members move on to Convolution2D layer.

Step 7 :Convolution generates feature maps.

Step 8 :A compilation method called MaxPooling2D that uses (2, 2) windows on a feature map that only retains the maximum number of pixels.
[3]

Step 9 :During training, Neural network Forward Propagation and back distribution are done at pixel values.

Step 10 :The Softmax function presents itself as an opportunity for each emotional class. The model is able to show the details of the possible emotional formation on the face.[2]

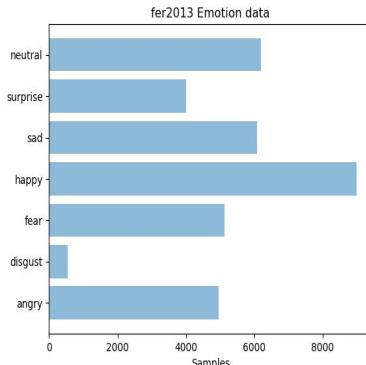
Database/ Dataset Specification

Three data sets are commonly used in different stages of the creation of this model.

- The model is initially fit on a training dataset, that is a set of examples used to fit the parameters (e.g. weights of connections between neurons in artificial neural networks) of the model. [10]
- The model (e.g. a neural net or a naive Bayes classifier) is trained on the training dataset using a supervised learning method (e.g. gradient descent or stochastic gradient descent). In practice, the training dataset often consist of pairs of an input vector and the corresponding answer vector or scalar, which is commonly denoted as the target. [4]
- The current model is run with the training dataset and produces a result, which is then compared with the target, for each input vector in the training dataset.
- Based on the result of the comparison and the specific learning algorithm being used, the parameters of the model are adjusted. The model fitting can include both variable selection and parameter estimation.

- Successively, the fitted model is used to predict the responses for the observations in a second dataset called the validation dataset.
- The validation dataset provides an unbiased evaluation of a model fit on the training dataset while tuning the model's hyperparameters (e.g. the number of hidden units in a neural network). [9]
- Validation datasets can be used for regularization by early stopping: stop training when the error on the validation dataset increases, as this is a sign of overfitting to the training dataset. This simple procedure is complicated in practice by the fact that the validation dataset's error may fluctuate during training, producing multiple local minima. This complication has led to the creation of many ad-hoc rules for deciding when overfitting has truly begun.[6]
- Finally, the test dataset is a dataset used to provide an unbiased evaluation of a final model fit on the training dataset.

- The data we used consists of 48x48 pixel grayscale images of faces.
- The faces have been automatically registered so that the face is more or less centred and occupies about the same amount of space in each image. [8]
- The task is to categorize each face based on the emotion shown in the facial expression into one of seven categories (0=Angry, 1=Disgust, 2=Fear, 3=Happy, 4=Sad, 5=Surprise, 6=Neutral).
- The training set consists of 28,709 examples and the public test set consists of 3,589 examples.



Emotion labels in the dataset:

- 0: -4593 images- Angry
- 1: -547 images- Disgust
- 2: -5121 images- Fear
- 3: -8989 images- Happy
- 4: -6077 images- Sad
- 5: -4002 images- Surprise
- 6: -6198 images- Neutral



Library and Packages

The Library Packages :

- OpenCV
- Numpy
- Numpy Array
- SciPy
- The Python Alternative To Matlab
- Keras
- TensorFlow

- **SYS :**

System-specific parameters and functions. This module provides access to some variables used or maintained by the interpreter and to functions that interact strongly with the interpreter. Using `help(sys)` provides valuable detail information.

- **Sigmoid Function :**

The sigmoid function in a neural network will generate the end point (activation) of inputs multiplied by their weights.

Softmax Function :

- Softmax function calculates the probabilities distribution of the event over n different events. In general way of saying, this function will calculate the probabilities of each target class over all possible target classes.

- The main advantage of using Softmax is the output probabilities range. The range will 0 to 1, and the sum of all the probabilities will be equal to one. If the softmax function used for multi-classification model it returns the probabilities of each class and the target class will have the high probability.
- The formula computes the exponential (e-power) of the given input value and the sum of exponential values of all the values in the inputs. Then the ratio of the exponential of the input value and the sum of exponential values is the output of the softmax function.

Experimental Results

The following emotions are the output as given below:

Emotion-recognition-master

```
File Edit Format Run Options Window Help
real_time_video.py - C:\Users\Archi\Downloads\Emotion-recognition-master\Emotion-recog...
from keras.preprocessing.image import img_to_array
import cv2
from keras.models import load_model
import numpy as np

# parameters for loading data and images
detection_model_path = 'haarcascade_files\haarcascade_frontalface_default.xml'
emotion_model_path = 'models\mini_XCEPTION.102-0.66.hdf5'

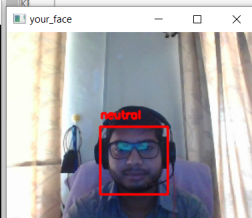
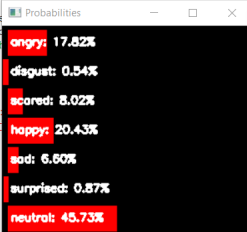
# hyper-parameters for bounding boxes shape
# loading models
face_detection = cv2.CascadeClassifier(detection_model_path)
emotion_classifier = load_model(emotion_model_path)
EMOTIONS = ["angry", "disgust", "scared", "happy", "sad", "surprised", "neutral"]

# feelings_faces = []
# for index, emotion in enumerate(EMOTIONS):
#     feelings_faces.append(cv2.imread('emo' + emotion + '.png'))

# starting video streaming
cv2.namedWindow('your_face')
camera = cv2.VideoCapture(0)
while True:
    frame = camera.read()[1]
    # reading the frame
    height, width, depth = frame.shape
    imgScale = 300 / width
    frame = cv2.resize(frame, None, fx=imgScale, fy=imgScale)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_detection.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5,
    canvas = np.zeros((250, 300, 3), dtype="uint8")
    frameClone = frame.copy()
    if len(faces) > 0:
        faces = sorted(faces, reverse=True,
```

IDLE Shell 3.10.2

```
File Edit Shell Debug Options Window Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17, 2023) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: C:\Users\Archi\Downloads\Emotion-recognition-master\real_time_video.py
```



Experimental Results

Emotion-recognition-master

File Home Share View

```
real_time_video.py - C:\Users\Archi\Downloads\Emotion-recognition-master\Emotion-recog...
File Edit Format Run Options Window Help
from keras.preprocessing.image import img_to_array
import cv2
from keras.models import load_model
import numpy as np

# parameters for loading data and images
detection_model_path = 'haarcascade_files\haarcascade_frontalface_default.xml'
emotion_model_path = 'models\mini_XCEPTION.102-0.66.hdf5'

# hyper-parameters for bounding boxes shape
# loading models
face_detection = cv2.CascadeClassifier(detection_model_path)
emotion_classifier = load_model(emotion_model_path)
EMOTIONS = ["angry", "disgust", "scared", "neutral"]

# feelings_faces = []
# for index, emotion in enumerate(EMOTIONS):
#     feelings_faces.append(cv2.imread('emo' + emotion + '.jpg'))

# starting video streaming
cv2.namedWindow('your_face')
camera = cv2.VideoCapture(0)
while True:
    frame = camera.read()[1]
    # reading the frame
    height, width, depth = frame.shape
    imgScale = 300 / width
    frame = cv2.resize(frame, None, fx=imgScale, fy=imgScale)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_detection.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5,
    minSize=30, flags=cv2.CASCADE_SCALE_IMAGE)

    canvas = np.zeros((250, 300, 3), dtype="uint8")
    frameClone = frame.copy()
    if len(faces) > 0:
        faces = sorted(faces, reverse=True,
        key=lambda x: (x[2] - x[0]) * (x[3] - x[1]))[0]
        (fx, fy, fw, fh) = faces
        # Extract the ROI of the face from the grayscale image, resi
```

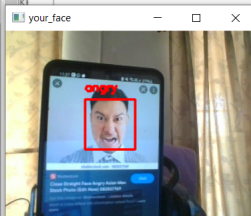
Probabilities

angry:	62.98%
disgust:	0.49%
scared:	0.83%
happy:	31.28%
sad:	0.04%
surprised:	3.02%
neutral:	1.36%

IDLE Shell 3.10.2

File Edit Shell Debug Options Window Help

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17, 2023) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: C:\Users\Archi\Downloads\Emotion-recognition-master\real_time_video.py
```



Experimental Results

Emotion-recognition-master

File Home Share View

```
real_time_video.py - C:\Users\Archi\Downloads\Emotion-recognition-master\Emotion-recog...
File Edit Format Run Options Window Help
from keras.preprocessing.image import img_to_array
import cv2
from keras.models import load_model
import numpy as np

# parameters for loading data and images
detection_model_path = 'haarcascade_files/haarcascade_frontalface_default.xml'
emotion_model_path = 'models/_mini_XCEPTION.102-0.66.hdf5'

# hyper-parameters for bounding boxes shape
# loading models
face_detection = cv2.CascadeClassifier(detection_model_path)
emotion_classifier = load_model(emotion_model_path)
EMOTIONS = ["angry", "disgust", "scared", "neutral"]

# feelings_faces = []
# for index, emotion in enumerate(EMOTIONS):
#     feelings_faces.append(cv2.imread('emo' + emotion + '.png'))

# starting video streaming
cv2.namedWindow('your_face')
camera = cv2.VideoCapture(0)
while True:
    frame = camera.read()[1]
    # reading the frame
    height,width,depth=frame.shape
    imgScale=300/width
    frame = cv2.resize(frame,None,fx=imgScale,fy=imgScale)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_detection.detectMultiScale(gray,scaleFactor=1.1,minNeighbors=5,

    canvas = np.zeros((250, 300, 3), dtype="uint8")
    frameClone = frame.copy()
    if len(faces) > 0:
        faces = sorted(faces, reverse=True,
            key=lambda x: (x[2] - x[0]) * (x[3] - x[1]))[0]
        (fx, fy, fw, fh) = faces
        # Extract the ROI of the face from the grayscale image, resi
```

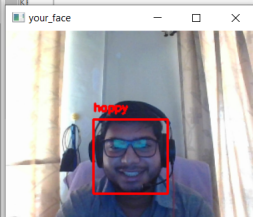
Probabilities

angry:	1.53%
disgust:	0.04%
scared:	1.10%
happy:	92.31%
sad:	0.73%
surprised:	0.08%
neutral:	4.21%

IDLE Shell 3.10.2

File Edit Shell Debug Options Window Help

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 15, 2023) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: C:\Users\Archi\Downloads\Emotion-recognition-master\real_time_video.py
```



Experimental Results

Emotion-recognition-master

File Home Share View

```
real_time_video.py - C:\Users\Archi\Downloads\Emotion-recognition-master\Emotion-recog...
File Edit Format Run Options Window Help
from keras.preprocessing.image import img_to_array
import cv2
from keras.models import load_model
import numpy as np

# parameters for loading data and images
detection_model_path = 'haarcascade_files/haarcascade_frontalface_default.xml'
emotion_model_path = 'models/_mini_XCEPTION.102-0.66.hdf5'

# hyper-parameters for bounding boxes shape
# loading models
face_detection = cv2.CascadeClassifier(detection_model_path)
emotion_classifier = load_model(emotion_model_path)
EMOTIONS = ["angry", "disgust", "scared", "happy", "sad", "surprised", "neutral"]

# feelings_faces = []
# for index, emotion in enumerate(EMOTIONS):
#     feelings_faces.append(cv2.imread('emo' + emotion + '.png'))

# starting video streaming
cv2.namedWindow('your_face')
camera = cv2.VideoCapture(0)
while True:
    frame = camera.read()[1]
    #reading the frame
    height,width,depth=frame.shape
    imgScale=300/width
    frame = cv2.resize(frame,None,fx=imgScale,fy=imgScale)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_detection.detectMultiScale(gray,scaleFactor=1.1,minNeighbors=5,

    canvas = np.zeros((250, 300, 3), dtype="uint8")
    frameClone = frame.copy()
    if len(faces) > 0:
        faces = sorted(faces, reverse=True,
            key=lambda x: (x[2] - x[0]) * (x[3] - x[1]))[0]
        (fx, fy, fw, fh) = faces
        # Extract the ROI of the face from the grayscale image, resi
```

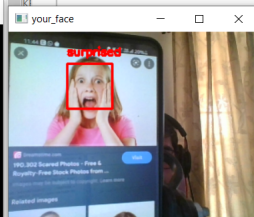
Probabilities

angry:	6.27%
disgust:	0.36%
scared:	14.67%
happy:	30.67%
sad:	7.65%
surprised:	37.12%
neutral:	3.25%

IDLE Shell 3.10.2

File Edit Shell Debug Options Window Help

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17, 2023) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: C:\Users\Archi\Downloads\Emotion-recognition-master\real_time_video.py
```



Experimental Results

Emotion-recognition-master

File Home Share View

```
real_time_video.py - C:\Users\Archi\Downloads\Emotion-recognition-master\Emotion-recog...
File Edit Format Run Options Window Help
from keras.preprocessing.image import img_to_array
import cv2
from keras.models import load_model
import numpy as np

# parameters for loading data and images
detection_model_path = 'haarcascade_files\haarcascade_frontalface_default.xml'
emotion_model_path = 'models/_mini_XCEPTION.102-0.66.hdf5'

# hyper-parameters for bounding boxes shape
# loading models
face_detection = cv2.CascadeClassifier(detection_model_path)
emotion_classifier = load_model(emotion_model_path)
EMOTIONS = ["angry", "disgust", "scared", "neutral"]

# feelings_faces = []
# for index, emotion in enumerate(EMOTIONS):
#     feelings_faces.append(cv2.imread('emo' + emotion + '.jpg'))

# starting video streaming
cv2.namedWindow('your_face')
camera = cv2.VideoCapture(0)
while True:
    frame = camera.read()[1]
    #reading the frame
    height,width,depth=frame.shape
    imgScale=300/width
    frame = cv2.resize(frame,None,fx=imgScale,fy=imgScale)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_detection.detectMultiScale(gray,scaleFactor=1.1,minNeighbors=5,

    canvas = np.zeros((250, 300, 3), dtype="uint8")
    frameClone = frame.copy()
    if len(faces) > 0:
        faces = sorted(faces, reverse=True,
            key=lambda x: (x[2] - x[0]) * (x[3] - x[1]))[0]
        (fx, fy, fw, fh) = faces
        # Extract the ROI of the face from the grayscale image, resi
```

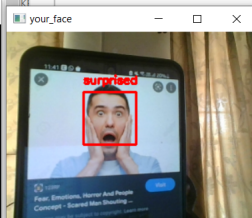
Probabilities

angry:	3.26%
disgust:	0.03%
scared:	12.56%
happy:	0.61%
sad:	1.97%
surprised:	80.55%
neutral:	1.00%

IDLE Shell 3.10.2

File Edit Shell Debug Options Window Help

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan
AMD64)] on win32
Type "help", "copyright", "credits" or "l
>>>
= RESTART: C:\Users\Archi\Downloads\Emoti
on-master\real_time_video.py
```



Performance Evaluation I

```
train_emotion_classifier.py - C:\Users\Arch\Downloads\Emotion-recognition-master\Emotion-recognition-master
File Edit Format Run Options Window Help

Description: Train emotion classification model
Description: Train emotion classification model
Description: Train emotion classification model

from keras.callbacks import CSVLogger, ModelCheckpoint, EarlyStopping
from keras.callbacks import ReduceLROnPlateau
from keras.preprocessing.image import ImageDataGenerator
from load_and_process import load_fer2013
from load_and_process import preprocess_input
from models.cnn import mini_XCEPTION
from sklearn.model_selection import train_test_split

# parameters
batch_size = 32
num_epochs = 10
input_shape = (48, 48, 1)
validation_split = .2
verbose = 1
num_classes = 7
patience = 50
base_path = 'models/'

# data generator
data_generator = ImageDataGenerator(
    featurewise_center=False,
    featurewise_std_normalization=False,
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=.1,
    horizontal_flip=True)

# model parameters/compilation
model = mini_XCEPTION(input_shape, num_classes)
model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])
model.summary()

# callbacks
log_file_path = base_path + '_emotion_training.log'
csv_logger = CSVLogger(log_file_path, append=False)
early_stop = EarlyStopping('val_loss', patience=patience)
reduce_lr = ReduceLROnPlateau('val_loss', factor=0.1,

IDLE Shell 3.10.2*
File Edit Shell Debug Options Window Help

conv2d_5 (Conv2D) (None, 3, 3, 128) 8192 ['add_2[0][0]']

max_pooling2d_3 (MaxPooling2D) (None, 3, 3, 128) 0 ['batch_normalization_1
3[0][0]']

batch_normalization_11 (BatchN (None, 3, 3, 128) 512 ['conv2d_5[0][0]']
ormalization)

add_3 (Add) (None, 3, 3, 128) 0 ['max_pooling2d_3[0][0]
',
'batch_normalization_1
1[0][0]']

conv2d_6 (Conv2D) (None, 3, 3, 7) 8071 ['add_3[0][0]']

global_average_pooling2d (Glob (None, 7) 0 ['conv2d_6[0][0]']
alAveragePooling2D)

predictions (Activation) (None, 7) 0 ['global_average_poolin
g2d[0][0]']

]

=====
Total params: 58,423
Trainable params: 56,951
Non-trainable params: 1,472
=====
```


Performance Evaluation II

```
Python 3.10.2 [tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15] [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: C:\Users\Archil\Downloads\Emotion-recognition-master\train_emotion_classifier.py
Model: "model"

Layer (type)                 Output Shape          Param #          Connected to
-----
input_1 (InputLayer)         [(None, 48, 48, 1)]  0                []
conv2d (Conv2D)              (None, 46, 46, 8)    72               ['input_1[0][0]']
batch_normalization (BatchNorm
alization)                  (None, 46, 46, 8)    32               ['conv2d[0][0]']
activation (Activation)       (None, 46, 46, 8)    0                ['batch_normalization[0][0]']
conv2d_1 (Conv2D)            (None, 44, 44, 8)    576              ['activation[0][0]']
batch_normalization_1 (BatchNo
rmalization)                 (None, 44, 44, 8)    32               ['conv2d_1[0][0]']
activation_1 (Activation)     (None, 44, 44, 8)    0                ['batch_normalization_1[0][0]']
separable_conv2d (SeparableCon
v2D)                         (None, 44, 44, 16)   200              ['activation_1[0][0]']
batch_normalization_3 (BatchNo
rmalization)                 (None, 44, 44, 16)   64               ['separable_conv2d[0][0]']
activation_2 (Activation)     (None, 44, 44, 16)   0                ['batch_normalization_3[0][0]']
separable_conv2d_1 (SeparableC
onv2D)                      (None, 44, 44, 16)   400              ['activation_2[0][0]']
batch_normalization_4 (BatchNo
rmalization)                 (None, 44, 44, 16)   64               ['separable_conv2d_1[0][0]']
conv2d_2 (Conv2D)            (None, 22, 22, 16)   128              ['activation_1[0][0]']
max_pooling2d (MaxPooling2D)  (None, 22, 22, 16)   0                ['batch_normalization_4[0][0]']
batch_normalization_2 (BatchNo
rmalization)                 (None, 22, 22, 16)   64               ['conv2d_2[0][0]']
add (Add)                    (None, 22, 22, 16)   0                ['max_pooling2d[0][0]',
'batch_normalization_2[0][0]']
```

Performance Evaluation III

```

IDLE Shell 3.10.2*
File Edit Shell Debug Options Window Help

max_pooling2d_2 (MaxPooling2D) (None, 6, 6, 64) 0 ['batch_normalization_10[0][0]']
batch_normalization_8 (Batch Normalization) (None, 6, 6, 64) 256 ['conv2d_4[0][0]']
add_2 (Add) (None, 6, 6, 64) 0 ['max_pooling2d_2[0][0]', 'batch_normalization_8[0][0]']
separable_conv2d_6 (Separable Conv2D) (None, 6, 6, 128) 8768 ['add_2[0][0]']
batch_normalization_12 (Batch Normalization) (None, 6, 6, 128) 512 ['separable_conv2d_6[0][0]']
activation_5 (Activation) (None, 6, 6, 128) 0 ['batch_normalization_12[0][0]']
separable_conv2d_7 (Separable Conv2D) (None, 6, 6, 128) 17536 ['activation_5[0][0]']
batch_normalization_13 (Batch Normalization) (None, 6, 6, 128) 512 ['separable_conv2d_7[0][0]']
conv2d_5 (Conv2D) (None, 3, 3, 128) 8192 ['add_2[0][0]']
max_pooling2d_3 (MaxPooling2D) (None, 3, 3, 128) 0 ['batch_normalization_13[0][0]']
batch_normalization_11 (Batch Normalization) (None, 3, 3, 128) 512 ['conv2d_5[0][0]']
add_3 (Add) (None, 3, 3, 128) 0 ['max_pooling2d_3[0][0]', 'batch_normalization_11[0][0]']
conv2d_6 (Conv2D) (None, 3, 3, 7) 8071 ['add_3[0][0]']
global_average_pooling2d (Global Average Pooling2D) (None, 7) 0 ['conv2d_6[0][0]']
predictions (Activation) (None, 7) 0 ['global_average_pooling2d[0][0]']

-----
Total params: 58,423
Trainable params: 56,951
Non-trainable params: 1,472

```

Ln 134 Col 0

Comparison with existing system

:

Comparison Chart: Market vs Our Model	
There are different models present in the market for emotion classification that uses graph neural network or capsule neural network	Our Approach: We used the convolutional neural network (CNN) because compared to its predecessors is that it automatically detects the important features without any human supervision.
Humans try to perceive and understand the emotion of others by their own mental filters and feelings	Emotion Recognition System: It is achieved by the capacity to read, understand and learn about emotional life of humans
BAUM is a video-based dataset. The dataset contains audio-visual clips of different languages and they are annotated. The clips represent real-world scenarios like different poses, lighting conditions, and subjects of varying ages. An image-based dataset is created with the peak frames from each clip. The BAUM-2i is an unconstrained expression database, whose samples are collected from movies and TV series. 998 images labeled with seven expressions are selected for evaluation.	FER2013 is a well-studied dataset. It is one of the more challenging datasets with human-level accuracy only at $65 \pm 5\%$ and the highest performing published works achieving 75.2% test accuracy. Easily downloadable on Kaggle, the dataset's 35,887 contained images are normalized to 48x48 pixels in grayscale. It contains 7 facial expressions, with distributions of Angry (4,953), Disgust (547), Fear (5,121), Happy (8,989), Sad (6,077), Surprise (4,002), and Neutral (6,198)



References I



Roddy Cowie, Ellen Douglas-Cowie, Nicolas Tsapatsoulis, George Votsis, Stefanos Kollias, Winfried Fellenz, and John G Taylor. Emotion recognition in human-computer interaction. *IEEE Signal processing magazine*, 18(1):32–80, 2001.



Moataz El Ayadi, Mohamed S Kamel, and Fakhri Karray. Survey on speech emotion recognition: Features, classification schemes, and databases. *Pattern recognition*, 44(3):572–587, 2011.



Alvin I Goldman and Chandra Sekhar Sripada. Simulationist models of face-based emotion recognition. *Cognition*, 94(3):193–213, 2005.

References II



Byoung Chul Ko.

A brief review of facial emotion recognition based on visual information.

sensors, 18(2):401, 2018.



Shashidhar G Koolagudi and K Sreenivasa Rao.

Emotion recognition from speech: a review.

International journal of speech technology, 15(2):99–117, 2012.



Ronak Kosti, Jose M Alvarez, Adria Recasens, and Agata Lapedriza.

Emotion recognition in context.

In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1667–1675, 2017.

References III



Emily Mower, Maja J Matarić, and Shrikanth Narayanan.

A framework for automatic human emotion classification using emotion profiles.

IEEE Transactions on Audio, Speech, and Language Processing, 19(5):1057–1070, 2010.



Björn Schuller, Gerhard Rigoll, and Manfred Lang.

Hidden markov model-based speech emotion recognition.

In *2003 IEEE International Conference on Acoustics, Speech, and Signal Processing, 2003. Proceedings.(ICASSP'03).*, volume 2, pages 11–1. Ieee, 2003.

References IV



Ryoko Tokuhsa, Kentaro Inui, and Yuji Matsumoto.

Emotion classification using massive examples extracted from the web.

In Proceedings of the 22nd International Conference on Computational Linguistics (Coling 2008), pages 881–888, 2008.



Suraj Tripathi, Abhay Kumar, Abhiram Ramesh, Chirag Singh, and Promod Yenigalla.

Deep learning based emotion recognition system using speech features and transcriptions.

arXiv preprint arXiv:1906.05681, 2019.